

Lab 2 Extra Practice Problems

Eighteen original exercises in reading x86-64 assembly, from easy to hard

How to Use This Packet

1. Each problem gives a function's signature and the assembly `gcc -O1` made from it. Write C with the same signature that behaves identically.
2. Read it the way Lab 2 teaches: arguments, then the loop (a backward jump), then a name for every register that changes, then the guards, then one sentence.
3. Trace one small input by hand before writing C, and test zero, negative values, and empty inputs after.
4. Use Part Two when stuck; open Part Three only after writing and testing an attempt.

Lab Assumptions

x86-64 Linux, AT&T syntax: source first, destination last. Arguments arrive in `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`; the result leaves in `%rax` (`%eax` for an `int`). `int` is 32 bits, `long` and pointers are 64 bits, `char` is 8 bits. Every listing is real `gcc -O1` output; assembler directives are removed and labels renumbered `.L1`, `.L2`, ... within each function. The few instructions not covered in Lectures 6–7 are explained where they appear.

On Linux you can check yourself: `gcc -O1 -fcf-protection=none -S -o - yours.c` often gives back the same listing. If it does not, your C may still be right, so trace both on the same input.

Part One Problems

1 Easy Exercises

1 Three-Way Add

long add3(long a, long b, long c)
What does it compute? Write C that behaves identically.

Look for: which argument is in which register; an `leaq` with no scale

From: Lecture 6

```
add3:
    addq    %rsi, %rdi
    leaq    (%rdi,%rdx), %rax
    ret
```

2 A Scaled Offset

int affine(int x)
What does it compute? Write C that behaves identically.

Look for: one `leaq` with a scale and a displacement

From: Lecture 6

```
affine:
    leal    4(%rdi,%rdi,8), %eax
    ret
```

3 Multiply Without a Multiply

int times12(int x)
What does it compute? Write C that behaves identically.

Look for: an `leaq`, then a shift

From: Lecture 6

```
times12:
    leal    (%rdi,%rdi,2), %eax
    sall    $2, %eax
    ret
```

4 Offsets Are Bytes

long first_plus_third(const long *p)
What does it compute? Write C that behaves identically.

Look for: a displacement on a pointer; which element it names

From: Lecture 6

```
first_plus_third:
    movq    (%rdi), %rax
    addq    16(%rdi), %rax
    ret
```

5 Two Loads, Two Stores

```
void swap(long *a, long *b)
```

What does it compute? Write C that behaves identically.

Look for: which moves read memory and which write it

From: Lecture 6

```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
    ret
```

6 Mixed Widths

```
int byte_sum(unsigned char a, signed char b)
```

What does it compute? Write C that behaves identically.

Look for: movzbl versus movsbl

From: Lecture 6

```
byte_sum:
    movzbl  %dil, %edi
    movsbl  %sil, %esi
    leal    (%rdi,%rsi), %eax
    ret
```

2 Medium Exercises

1 Pick the Larger

```
int max2(int a, int b)
```

What does it compute? Write C that behaves identically.

Look for: `cmov`; the operand order of `cmp`

From: Lecture 7

```
max2:
    cmpl    %edi, %esi
    movl    %edi, %eax
    cmovge  %esi, %eax
    ret
```

2 Signed or Unsigned?

```
int is_below(unsigned a, unsigned b)
```

What does it compute? Write C that behaves identically.

Look for: `setb`, which sets one byte to 1 if the last compare was *below* **From:** Lecture 7; `setb` is new and to 0 otherwise

```
is_below:
    cmpl    %esi, %edi
    setb    %al
    movzbl  %al, %eax
    ret
```

3 Compute Both, Keep One

```
int abs_diff(int a, int b)
```

What does it compute? Write C that behaves identically.

Look for: two subtractions before the compare

From: Lecture 7

```
abs_diff:
    movl    %edi, %edx
    subl    %esi, %edx
    movl    %esi, %eax
    subl    %edi, %eax
    cmpl    %esi, %edi
    cmovg   %edx, %eax
    ret
```

4 A Counted Loop

```
int power(int base, int exp)
```

What does it compute? Write C that behaves identically.

Look for: `imull` `s`, `d`, which is `d = d * s`; the guard before the loop **From:** Lecture 7; `imull` is new

```
power:
    testl    %esi, %esi
    jle      .L1
    movl     $0, %eax
    movl     $1, %edx
.L2:
    imull    %edi, %edx
    addl     $1, %eax
    cmpl     %eax, %esi
    jne      .L2
.L3:
    movl     %edx, %eax
    ret
.L1:
    movl     $1, %edx
    jmp      .L3
```

5 Search

```
int find(const int *a, int n, int key)
```

What does it compute? Write C that behaves identically.

Look for: two different ways out of the loop **From:** Lecture 7

```
find:
    testl    %esi, %esi
    jle      .L1
    movslq   %esi, %rsi
    movl     $0, %eax
.L2:
    cmpl     %edx, (%rdi,%rax,4)
    je       .L3
    addq     $1, %rax
    cmpq     %rsi, %rax
    jne      .L2
    movl     $-1, %eax
    ret
.L1:
    movl     $-1, %eax
.L3:
    ret
```

6 The Counter Counts Bytes

```
int dot(const int *a, const int *b, int n)
```

What does it compute? Write C that behaves identically.

Look for: an index that goes up by 4; `imull` with a memory source **From:** Lecture 7; `imull` is new

```
dot:
    testl    %edx, %edx
    jle      .L1
    movslq   %edx, %rdx
    leaq     0(,%rdx,4), %r8
    movl     $0, %eax
    movl     $0, %ecx
.L2:
    movl     (%rdi,%rax), %edx
    imull    (%rsi,%rax), %edx
    addl     %edx, %ecx
    addq     $4, %rax
    cmpq     %r8, %rax
    jne      .L2
.L3:
    movl     %ecx, %eax
    ret
.L1:
    movl     $0, %ecx
    jmp      .L3
```

3 Hard Exercises

1 Division That Rounds Toward Zero

```
int div8(int x)
```

What does it compute? Write C that behaves identically.

Look for: `sarl` rounds down; what the `+7` is for

From: Lecture 6–7; Lab 1

```
div8:
    leal    7(%rdi), %eax
    testl   %edi, %edi
    cmovns  %edi, %eax
    sarl    $3, %eax
    ret
```

2 A Row of Five

```
int grid_get(int g[][5], long i, long j)
```

What does it compute? Write C that behaves identically.

Look for: two `leas` that build an address, then one load

From: Lecture 6

```
grid_get:
    leaq    (%rsi,%rsi,4), %rax
    leaq    (%rdi,%rax,4), %rax
    movl    (%rax,%rdx,4), %eax
    ret
```

3 Following Pointers

```
long list_sum(const struct node *p)
```

What does it compute? Write C that behaves identically.

Look for: `8(%rdi)` is a field; `testq` is a NULL check. Here `struct node { long val; struct node *next; }` **From:** Lecture 6–7

```
list_sum:
    testq   %rdi, %rdi
    je      .L1
    movl    $0, %eax
.L2:
    addq    (%rdi), %rax
    movq    8(%rdi), %rdi
    testq   %rdi, %rdi
    jne     .L2
    ret
.L1:
    movl    $0, %eax
    ret
```

4 Walking Two Strings

```
int str_eq(const char *a, const char *b)
```

What does it compute? Write C that behaves identically.

Look for: two pointers that advance together; **sete**, the *equal* version **From:** Lecture 7; **sete** is new of **setb**

```
str_eq:
    movzbl    (%rdi), %eax
    testb     %al, %al
    je        .L1
.L2:
    cmpb      %al, (%rsi)
    jne       .L1
    addq      $1, %rdi
    addq      $1, %rsi
    movzbl    (%rdi), %eax
    testb     %al, %al
    jne       .L2
.L1:
    cmpb      %al, (%rsi)
    sete      %al
    movzbl    %al, %eax
    ret
```

5 A Jump Table

int apply(int op, int x)

What does it compute? Write C that behaves identically.

Look for: a table of offsets, not addresses; the same shape as Lab 2 bomb phase 3 **From:** new: jump tables (CS:APP §3.6.8)

```

apply:
    cmpl    $5, %edi
    ja      .L1
    movl    %edi, %edi
    leaq    .L2(%rip), %rdx
    movslq  (%rdx,%rdi,4), %rax
    addq    %rdx, %rax
    jmp     *%rax
.L2:
    .long   .L3-.L2
    .long   .L4-.L2
    .long   .L5-.L2
    .long   .L6-.L2
    .long   .L7-.L2
    .long   .L8-.L2
.L3:
    leal    1(%rsi), %eax
    ret
.L4:
    leal    (%rsi,%rsi,2), %eax
    ret
.L5:
    leal    -7(%rsi), %eax
    ret
.L6:
    leal    0(,%rsi,4), %eax
    ret
.L7:
    movl    %esi, %eax
    xorl    $5, %eax
    ret
.L8:
    movl    %esi, %eax
    negl    %eax
    ret
.L1:
    movl    $0, %eax
    ret

```

6 Recursion and Saved Registers

```
int fib(int n)
```

What does it compute? Write C that behaves identically.

Look for: pushq/popq of %rbx and %rbp; two calls

From: Lecture 7

```
fib:
    movl    %edi, %eax
    cmpl    $1, %edi
    jle     .L1
    pushq   %rbp
    pushq   %rbx
    subq    $8, %rsp
    movl    %edi, %ebx
    leal    -1(%rdi), %edi
    call    fib
    movl    %eax, %ebp
    leal    -2(%rbx), %edi
    call    fib
    addl    %ebp, %eax
    addq    $8, %rsp
    popq    %rbx
    popq    %rbp
    ret
.L1:
    ret
```

Part Two Explanations and Hints

4 Easy Exercises

1 Three-Way Add

First hint. Name the three argument registers before reading anything else: `%rdi`, `%rsi`, `%rdx`. **Why the idea works.** `addq %rsi, %rdi` leaves $a + b$ in `%rdi`. `leaq (%rdi,%rdx), %rax` adds c and puts the total in the return register without touching memory.

2 A Scaled Offset

First hint. Expand $D(B, I, S)$ as $B + I \cdot S + D$, with the real registers and numbers filled in. **Why the idea works.** `4(%rdi,%rdi,8)` is $x + 8x + 4 = 9x + 4$. Using the same register as base and index is how one `lea` multiplies by 3, 5 or 9.

3 Multiply Without a Multiply

First hint. Write down what `%eax` holds after each of the two instructions, separately. **Why the idea works.** The `lea` gives $3x$ and `sall $2` multiplies by 4, so the result is $12x$. Compilers build small multiplies out of `lea` and shifts because they are cheaper than a multiply.

4 Offsets Are Bytes

First hint. A `long` is 8 bytes. How many elements into the array is byte 16? **Why the idea works.** `(%rdi)` is `p[0]` and `16(%rdi)` is two 8-byte elements in, `p[2]`. `addq` can take its source straight from memory.

5 Two Loads, Two Stores

First hint. Parentheses on the *destination* side of a `mov` mean a store. **Why the idea works.** Both values are loaded into registers before either is stored, so neither is lost. The temporary a C programmer would declare lives in `%rax`.

6 Mixed Widths

First hint. Which of the two extensions fills the new bits with zeros, and which copies the sign bit? **Why the idea works.** The `unsigned char` is zero-extended and the `signed char` sign-extended; once both are 32-bit values, the `lea` adds them. The C types chose the moves.

5 Medium Exercises

1 Pick the Larger

First hint. `cmpl %edi, %esi` computes `%esi - %edi`, that is, $b - a$. **Why the idea works.** `%eax` starts as `a`, and `cmovge` replaces it with `b` when $b \geq a$. On a tie it returns `b`, which is the same value, so it matches `a > b ? a : b`.

2 Signed or Unsigned?

First hint. The `b` in `setb` is the same *below* as in `jb`, from the unsigned family. **Why the idea works.** `cmpl %esi, %edi` computes $a - b$, and *below* means $a < b$ as unsigned numbers. `setb` writes only `%al`, so `movzbl` clears the rest of `%eax`.

3 Compute Both, Keep One

First hint. Name `%edx` and `%eax` after the first four instructions, before you look at the `cmov`. **Why the idea works.** `%edx = a - b` and `%eax = b - a`. `cmovg` keeps $a - b$ when $a > b$. Computing both answers and choosing one avoids a branch.

4 A Counted Loop

First hint. `%eax` counts and `%edx` accumulates. What comes back if the loop never runs? **Why the idea works.** The result starts at 1 and is multiplied by `base` once per trip, `exp` times. When $\text{exp} \leq 0$ the guard skips the loop and returns 1.

5 Search

First hint. The `je` inside the loop leaves with `%eax` still holding the index. **Why the idea works.** `%rax` is `i` and `(%rdi,%rax,4)` is `a[i]`. A match returns `i`; running off the end returns `-1`, and so does $n \leq 0$.

6 The Counter Counts Bytes

First hint. `%rax` steps by 4 and the loop stops when it reaches `%r8`. What is `%r8`? **Why the idea works.** The compiler scaled the counter instead of the address: `%rax = 4i`, so `(%rdi,%rax)` is `a[i]`. Each trip multiplies `a[i]` by `b[i]` and adds it to `%ecx`.

6 Hard Exercises

1 Division That Rounds Toward Zero

First hint. Try $x = -1$ with and without the `+7`. What should $-1 / 8$ be in C? **Why the idea works.** `sarl $3` is $\text{floor}(x / 8)$, but C division truncates toward zero. For negative x the compiler adds 7 first; `cmovns` keeps plain x when $x \geq 0$. It is Lab 1's `dividePower2` bias, done by the compiler.

2 A Row of Five

First hint. Each row is five ints, 20 bytes. Where does row i start? **Why the idea works.** The first `lea` gives $5i$ and the second `g + 4 · 5i`, the address of row i . `(%rax,%rdx,4)` then indexes j within it.

3 Following Pointers

First hint. At offset 0 of a node is `val`, at offset 8 is `next`. Which one is loaded back into `%rdi`? **Why the idea works.** Each trip adds `p->val`, then replaces `p` with `p->next`. The loop ends when `p` is NULL, and an empty list returns 0.

4 Walking Two Strings

First hint. The loop has two exits. Which is taken when `*a` is 0, and which when the bytes differ? Where do both go? **Why the idea works.** The loop runs while `*a` is nonzero and equal to `*b`. Both exits reach the same final `cmpb`, and `sete` returns 1 exactly when the two stopping bytes are equal. For equal strings, those are both terminators.

5 A Jump Table

First hint. `ja` is an unsigned compare. Where does `op = -1` go? Then read entry `op` of `.L2` and add `.L2` back. **Why the idea works.** Each `.long` holds `target - table`. `movslq` reads entry `op` and `addq` adds the table's address, rebuilding the target. The single unsigned `ja` sends both `op < 0` and `op > 5` to the default case.

6 Recursion and Saved Registers

First hint. Why are n and the first result kept in `%rbx` and `%rbp`, not left in `%edi` and `%eax`? **Why the idea works.** A `call` may overwrite caller-saved registers such as `%edi` and `%eax`. `%rbx` and `%rbp` are callee-saved, so the function saves them, uses them to carry n and $\text{fib}(n - 1)$ across the second call, and restores them. `subq $8` keeps `%rsp` 16-byte aligned at each call.

Part Three Complete Solutions

7 Easy Exercises

1 Three-Way Add

```
long add3(long a, long b, long c) {
    return a + b + c;
}
```

Key check. `addq %rsi, %rdi` leaves $a + b$ in `%rdi`. `leaq (%rdi,%rdx), %rax` adds c and puts the total in the return register without touching memory.

2 A Scaled Offset

```
int affine(int x) {
    return 9 * x + 4;
}
```

Key check. `4(%rdi,%rdi,8)` is $x + 8x + 4 = 9x + 4$. Using the same register as base and index is how one `leaq` multiplies by 3, 5 or 9.

3 Multiply Without a Multiply

```
int times12(int x) {
    return x * 12;
}
```

Key check. The `leaq` gives $3x$ and `salq $2` multiplies by 4, so the result is $12x$. Compilers build small multiplies out of `leaq` and shifts because they are cheaper than a multiply.

4 Offsets Are Bytes

```
long first_plus_third(const long *p) {
    return p[0] + p[2];
}
```

Key check. `(%rdi)` is $p[0]$ and `16(%rdi)` is two 8-byte elements in, $p[2]$. `addq` can take its source straight from memory.

5 Two Loads, Two Stores

```
void swap(long *a, long *b) {  
    long t = *a;  
    *a = *b;  
    *b = t;  
}
```

Key check. Both values are loaded into registers before either is stored, so neither is lost. The temporary a C programmer would declare lives in `%rax`.

6 Mixed Widths

```
int byte_sum(unsigned char a, signed char b) {  
    return a + b;  
}
```

Key check. The `unsigned char` is zero-extended and the `signed char` sign-extended; once both are 32-bit values, the `lea` adds them. The C types chose the moves.

8 Medium Exercises

1 Pick the Larger

```
int max2(int a, int b) {
    return a > b ? a : b;
}
```

Key check. `%eax` starts as `a`, and `cmovge` replaces it with `b` when $b \geq a$. On a tie it returns `b`, which is the same value, so it matches `a > b ? a : b`.

2 Signed or Unsigned?

```
int is_below(unsigned a, unsigned b) {
    return a < b;
}
```

Key check. `cmpl %esi, %edi` computes $a - b$, and *below* means $a < b$ as unsigned numbers. `setb` writes only `%al`, so `movzbl` clears the rest of `%eax`.

3 Compute Both, Keep One

```
int abs_diff(int a, int b) {
    if (a > b)
        return a - b;
    return b - a;
}
```

Key check. `%edx = a - b` and `%eax = b - a`. `cmovg` keeps $a - b$ when $a > b$. Computing both answers and choosing one avoids a branch.

4 A Counted Loop

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 0; i < exp; i++)
        result *= base;
    return result;
}
```

Key check. The result starts at 1 and is multiplied by `base` once per trip, `exp` times. When $\text{exp} \leq 0$ the guard skips the loop and returns 1.

5 Search

```
int find(const int *a, int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key)
            return i;
    return -1;
}
```

Key check. `%rax` is `i` and `(%rdi,%rax,4)` is `a[i]`. A match returns `i`; running off the end returns `-1`, and so does `n ≤ 0`.

6 The Counter Counts Bytes

```
int dot(const int *a, const int *b, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i] * b[i];
    return sum;
}
```

Key check. The compiler scaled the counter instead of the address: `%rax = 4i`, so `(%rdi,%rax)` is `a[i]`. Each trip multiplies `a[i]` by `b[i]` and adds it to `%ecx`.

9 Hard Exercises

1 Division That Rounds Toward Zero

```
int div8(int x) {
    return x / 8;
}
```

Key check. `sarl $3` is $\text{floor}(x / 8)$, but C division truncates toward zero. For negative x the compiler adds 7 first; `cmovns` keeps plain x when $x \geq 0$. It is Lab 1's `dividePower2` bias, done by the compiler.

2 A Row of Five

```
int grid_get(int g[][5], long i, long j) {
    return g[i][j];
}
```

Key check. The first `lea` gives $5i$ and the second `g + 4 · 5i`, the address of row i . `(%rax,%rdx,4)` then indexes j within it.

3 Following Pointers

```
struct node {
    long val;
    struct node *next;
};

long list_sum(const struct node *p) {
    long sum = 0;
    while (p) {
        sum += p->val;
        p = p->next;
    }
    return sum;
}
```

Key check. Each trip adds `p->val`, then replaces `p` with `p->next`. The loop ends when `p` is `NULL`, and an empty list returns 0.

4 Walking Two Strings

```
int str_eq(const char *a, const char *b) {
    while (*a && *a == *b) {
        a++;
        b++;
    }
    return *a == *b;
}
```

Key check. The loop runs while `*a` is nonzero and equal to `*b`. Both exits reach the same final `cmpb`, and `sete` returns 1 exactly when the two stopping bytes are equal. For equal strings, those are both terminators.

5 A Jump Table

```
int apply(int op, int x) {
    switch (op) {
        case 0: return x + 1;
        case 1: return x * 3;
        case 2: return x - 7;
        case 3: return x << 2;
        case 4: return x ^ 5;
        case 5: return -x;
        default: return 0;
    }
}
```

Key check. Each `.long` holds `target - table`. `movslq` reads entry `op` and `addq` adds the table's address, rebuilding the target. The single unsigned `ja` sends both `op < 0` and `op > 5` to the default case.

6 Recursion and Saved Registers

```
int fib(int n) {
    if (n < 2)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```

Key check. A call may overwrite caller-saved registers such as `%edi` and `%eax`. `%rbx` and `%rbp` are callee-saved, so the function saves them, uses them to carry `n` and `fib(n - 1)` across the second call, and restores them. `subq $8` keeps `%rsp` 16-byte aligned at each call.

Suggested Boundary Tests

Category	Values	Reason
Integers	0, 1, -1, INT_MIN, INT_MAX	Expose sign, overflow and cmp-order errors
Loop counts	$n = 0, 1$, and negative	Expose the guard before the loop
Division	-1, -7, -8, -9	Expose floor versus truncation
Pointers and strings	empty list, empty string, one string a prefix of the other	Expose the exits of a pointer loop
Switch	every case, -1, one past the last case	Expose the default path and the unsigned bound

Verification note. Every listing was regenerated from the Part Three solutions with gcc 13.3 and compared line for line, and every solution was run against the assembly printed in Part One on boundary cases and randomized inputs: 224,107 checks, none failed (`scripts/check-practice.py`).